WemiEnv: An Open-source Reinforcement Learning Platform for WeChat Mini-Games

Longxiang Shi *Member, IEEE*, Qianchen Ding, Jingzhe Hou, Binbin Zhou *Member, IEEE*, Canghong Jin *Member, IEEE*, Ye Tao, Jinling Wei, and Shijian Li *Member, IEEE*

***Abstract*—The popularity of mobile games has surged in recent years. Along with mobile games, the emergence of mini-games has recently raised attention. Compared to traditional mobile games, mini-games are more lightweight and platform-independent with low development cost, which has attracted thousands of developers and users. WeChat mini-games platform is one of the most popular platforms with over 100,000 mini-games. The diversity and variety of WeChat mini-games make it an ideal platform for training reinforcement learning (RL) agents. In contrast, most of the existing RL benchmark environments are equipped with predetermined games, which are always limited to several genres and lack the utilization of new and diverse games. To utilize the WeChat mini-games for RL research, in this paper, we propose WemiEnv, a lightweight, easy-to-use and open-source platform for reinforcement learning research towards WeChat mini-games. WemiEnv is built on the WeChat developer tools and allows RL agents to interact with the mini-games. WemiEnv also supports user-customized mini-games, requiring users to implement only a few interface functions within WemiEnv API. We also provide 6 popular mini-games: Space Fighter, Flip, 2048, Flappy Bird, Timberman and Snake as ready-to-use tasks. Experiments were conducted with the OpenAI Spinning Up library for RL baselines on the provided tasks to test the usability of WemiEnv. Our code and documentation are available at https://github.com/Ada9io/WemiEnv.**

***Index Terms*—deep reinforcement learning, games, neural networks, open-source platform**

I. Introduction

THE popularity of mobile games has surged among smartphone users in recent decades. More than 110 billion dollars in consumer spending on mobile games was reported for 2022[1]. The substantial profits have not only led to an increase in the production of new games but also to the emergence of multiple mini-game platforms that offer a variety of mini-games. In contrast to conventional mobile game apps, mini-games are integrated into mobile apps, enabling users to play them directly without downloading additional game apps. The appeal of this "plug-and-play" feature has drawn in a significant number of players, along with numerous developers and development platforms. Among the available mini-game platforms, the WeChat mini-game platform has emerged as one of the most popular. Developed by *Tencent*, it is integrated with the WeChat app, which has an enormous user base. Currently, the WeChat mini-game platform hosts over 100,000 mini-games and maintains a monthly user base of 400 million active users [29].

Recently, reinforcement learning (RL) [22] as a promising paradigm for artificial intelligence has achieved great success in game play. Prominent examples such as Atari [12], Go [20], [21], StarCraft [28], and Poker [13] have demonstrated the effectiveness in solving complex game tasks. To evaluate the performance of RL algorithms for different tasks, RL platforms were developed to facilitate researchers in evaluating their work through a unified interface. The Atari suite [4] is a pioneer and has been followed by a variety of platforms, such as *OpenAI* Gym[5] and Universe[2] and VizDoom [10]. The Gym interfaces have been widely adopted for RL research platforms. Although these existing RL platforms offer an easy way for researchers to assess the RL algorithms, most of them are focused on several certain game genres in some fixed games (e.g., StarCraft, Doom, MineCraft and Atari) and lack utilization for the new emerging games. In contrast, the WeChat mini-game platform is a thriving gaming platform that maintains a good ecosystem with low development cost, with thousands of games ranging from casual and social games to more intricate and engaging role-play games. Studying algorithms to tackle the new and diverse genres of mini-games will greatly facilitate RL research.

Therefore, to fully utilize the WeChat mini-games, in this paper, we propose WemiEnv, a lightweight, easy-to-use, and open-source reinforcement learning platform for WeChat mini-games. WemiEnv platform is built on WeChat developer tools[3] and provides *OpenAI* Gym style Application Programming Interface (API) based on Python. Our WemiEnv platform connects the WeChat mini-games and Python API through a socket server. The RL agent can interact with the mini-games through a stable link with low communication latency. WemiEnv also supports user-customized mini-games, requiring users to implement only a few simple interface functions themselves within the WemiEnv API. Based on WemiEnv, we then provide six popular mini-games as the initial set of ready-to-use tasks: Space Fighter, Flip, 2048, Flappy Bird,

L. Shi, Q. Ding, J. Hou, B. Zhou, C. Jin, and J. Wei are with the Department of Computer and Computing Science, Hangzhou City University, Hangzhou, China. Email: shilx@hzcu.edu.cn, 32103082@stu.hzcu.edu.cn, 32101045@stu.hzcu.edu.cn, bbzhou@hzcu.edu.cn, jinch@hzcu.edu.cn, weijl@hzcu.edu.cn. L. Shi and J. Wei are also with Zhengzhou Digital Industry Institute, Zhengzhou, China.

Y. Tao is with the Industrial Design Department of Hangzhou City University. Email: taoye@hzcu.edu.cn

S. Li is with the Department of Computer Science and Technology, Zhejiang University, China. Email: shijianli@zju.edu.cn

Corrensponding author: Canghong Jin(jinch@hzcu.edu.cn)

[1]https://www.gamesindustry.biz/dataai-global-mobile-game-revenue-dipped-5-to-110bn-in-2022

[2]https://openai.com/research/universe

[3]https://developers.weixin.qq.com/miniprogram/en/dev/devtools/devtools.html

Timberman and Snake for demonstration. We also present empirical results on the six mini-games with several deep RL methods, which are from the widely used RL baseline library Spinning Up[4] from *OpenAI*. Our code and documentation are available at GitHub: https://github.com/Ada9io/WemiEnv. To be brief, our contribution can be summarized as below:

- A new open-source platform is proposed to facilitate the research on the WeChat mini-games.
- We provide six initial mini-game tasks with the proposed WemiEnv platform.
- Experiments are conducted with several deep RL methods on the proposed methods to demonstrate the ease of use of WemiEnv.

II. Related Works and Preliminary

A. Related Works

As RL methods require an interactive environment to assess their performance, numerous existing platforms are developed to facilitate RL research. In 2016, Atari2600 [4] was the first platform for RL agents through a predefined observation and action interface. Torchcraft[23] is also developed by Facebook AI for training RL agents in StarCraft. *OpenAI*, as a leading company for RL research, has released several platforms for RL research. For example, the Gym environment is one of the most widely used toolkits for assessing the RL algorithms [5]. The Universe platform[5] provides Flash games, browser tasks, and complex games such as slither.io and GTA V. Additionally, Universe is integrated with some other RL environments such as World of Bits [19]. Google's *DeepMind* also developed several platforms for RL research. For example, the DeepMind Lab [3] provides a 3D environment that contains puzzle-solving and navigation tasks. AndroidEnv [26] is an open source RL platform for Android apps, which allows the RL agent to interact with the apps within the Android ecosystem. Dm-control [27] is a collection of Python libraries for RL in robotic simulation developed by *DeepMind*. In addition, the Malmo platform [8] is developed by *Microsoft* and can provide RL interfaces for the game Minecraft. The MuJoCo [25] platform is a widely used simulator for RL research, providing various robotic tasks. The Horizon platform is developed by *Facebook* and is an open-source applied RL platform that aims at solving industry-applied RL platforms, especially when the datasets are large.

Unlike the RL platforms developed above, which contain various tasks, the platforms developed from academia mostly focus on a specific domain. For example, the ViZDoom [10] is designed as a research platform for first-person shooter(FPS) games, which is lightweight and highly customizable. MAgent [33] is a multi-agent RL platform for artificial collective intelligence. SurRoL [31] provides a research platform for surgical robot learning. RLCard [32] is developed for RL evaluation in card games. RL-ABMS [6] provides a ride-sourcing platform for spatial-temporal pricing problems. DRLinFluids [30] provides an RL research platform for fluid mechanics with a Python interface. Amazon's DeepRacer [1], [2] provides an autonomous racing platform for experimentation with Sim2Real RL. Similarly, TMRL [6] and [14] also provide environments for racing games. ML-Agents [9] a platform that enables games and simulations to serve as environments for training intelligent agents. More recently, SofaGym [15] was developed for RL research on soft robot simulations. Most of those platforms adopted the *OpenAI* Gym style interface and facilitated the study on a certain domain.

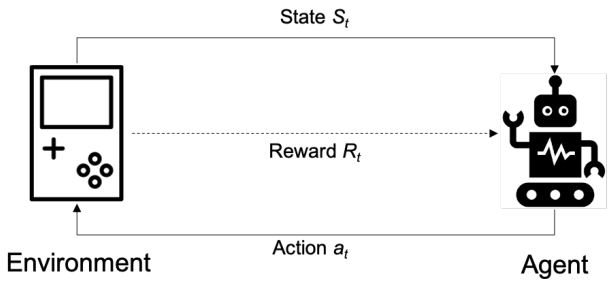


Fig. 1: Agent-environment RL interface

Unfortunately, although those existing RL benchmark platforms provided an easy way to evaluate of RL methods, they are mostly focused on several certain game genes in some fixed games and lack utilization for the new and diverse games. In this paper, we propose WemiEnv for RL researchers to utilize the new and diverse games in WeChat mini-game ecosystem. The comparison between WemiEnv and some of the existing RL benchmark platforms is shown in Table I. We compare the environments in five dimensions:

- Universal interface: whether the environment adopts a universal interface to facilitate users. The most popular is the OpenAI gym-style interface.
- Extensible task suite: Whether the environment supports self-defined tasks.
- Real-time communication protocol: Whether the agent interacts with the environments using real-time communication protocols.
- Plug-in game support: Whether the environment supports plug-in games.
- Package and game size: The size of the environment and the game installation package.

B. Preliminary

In reinforcement learning, an agent interacts with an environment, aiming to find an optimal policy to maximize the expectations of cumulative rewards obtained by the agent [22], [18]. Mathematically, the reinforcement learning process can be modeled as a Markov Decision Process (MDP) with the key components as $< S, A, P, R, \gamma >$ which can be defined as follows:

- **State** S: A state $s \in S$ is defined as the observations of the agent obtained from the environment.
- **Action** A: An action $a \in A$ is a signal sent from the agent to the environment, denoting the agent's decision in a specific state.

[4]https://spinningup.openai.com/en/latest
[5]https://openai.com/research/universe
[6]https://github.com/trackmania-rl/tmrl

TABLE I: Summary of the environment properties

Environment	Universal interface	Extensible task suite	Real-time communication protocol	Plug-in game support	Package size	Game size	Comment
Atari	✓				16.8MB	587KB for The Atari ROM files.	Based on the OpenAI Gym
TorchCraft	✓	✓	✓		46.2MB	1.10GB for the installation of StarCraft:Broodwar.	Based on StarCraft
MineCraft	✓	✓			4.31MB	1.12GB for the installation of MineCraft.	Based on MineCraft
RLCard	✓		✓		1.15MB	Package already contains the initial tasks.	
Magent	✓		✓		27.6MB	Package already contains the initial tasks.	
ViZDoom	✓	✓	✓		19.8MB	29.8MB for the original tasks.	Based on Doom
SofaGym	✓				64.7MB	Package already contains the initial tasks.	Based on the OpenAI Gym
OpenAI Gym	✓	✓			2.22MB	Package already contains the initial tasks.	
OpenAI Universe	✓	✓	✓		1.98MB	Package contains partial games	
AndroidEnv	✓	✓	✓	✓	185MB	Package already contains the initial tasks.	
WemiEnv	✓	✓	✓	✓	52.2KB	11.6MB for the initial 6 games.	Based on WeChat simulator

- **Transition probability function** P: $P(s'|s,a)$ is the transition function that indicates the new state s', when the agent executes action a at the state s.
- **Reward function** R: $R(s,a)$ is a function that figures out the immediate reward the environment provides when the agent executes action a under the state s.
- **Discount factor** γ: $\gamma \in [0,1]$ defines the discount factor that measures the value of present and future rewards.

The general agent-environment interface is shown in Fig. 1. At each timestep, the agent observes a state s_t, executes an action a_t based on some policy, and then receives a reward r_t from the environment based on the reward function R. Subsequently, the environment transits to a new state s_{t+1} based on the transition probability function P. Each episode in the MDP denotes an interaction process and the agent gets the discounted sum of rewards as the return. For an episode with a duration of T, the cumulative reward of the t-th step is: $G_t = \sum_{i=t}^{T}(\gamma^{i-t}R(s_t,a_t))$. The goal of RL is to find a policy $\pi : S \rightarrow A$ that maximizes the cumulative reward for the whole interactive process:

$$\pi = \arg\max_{a \in A} \mathbb{E}[G_t] \tag{1}$$

III. Environment Details

WemiEnv facilitates the interaction of RL agents, allowing them to learn and solve tasks with WeChat mini-games. In this section, we will describe the WemiEnv platform in detail. We will first introduce the overall architecture, followed by the real-time interactive framework and the environment interface. Then six support tasks are depicted. We also provide a latency analysis of the platform. Finally, the application programming interface is described.

A. Overall Architecture

The overall architecture of WemiEnv is shown in Fig 2. WemiEnv adopts the WebSocket API to interact with the WeChat mini-grames. Within this platform, WeChat minigames are required to run in the WeChat DevTools simulator[7]. We utilize the WebSocket API to communicate with the WeChat mini-games on the simulators. The WebSocket is a communication protocol that provides full-duplex communication channels over a single, long-lived connection. It allows efficient, bidirectional communication between clients and servers, making it well-suited for real-time applications where low-latency communication is essential. As the WeChat mini-games are developed in JavaScript, it is well-compatible with the WebSocket API. Therefore, we adopt the WebSocket API in WemiEnv to ensure efficient and reliable communication between WeChat mini-games and the RL agent.

The WemiEnv adopts client-server architecture and contains 3 modules: WebSocket client module, WebSocket server module, and Interface module.

- **WebSocket client** is designed for information exchange with the WeChat simulator. As the WeChat mini-games are developed in JavaScript, the WebSocket client module is also implemented in Javascript. It integrates with the WeChat mini-games and is responsible for transmitting the environmental information such as states, rewards, etc., from the WeChat mini-games to the WebSocket server module.
- **WebSocket Server** serves between the WebSocket client and the interface. It exchanges the information with the WebSocket client and processes it to the interface. It's implemented in Python.
- **Interface** provides a Gym-style, unified interface for the WeChat mini-games. It exchanges information with the

[7]https://developers.weixin.qq.com/miniprogram/en/dev/devtools/devtools.html

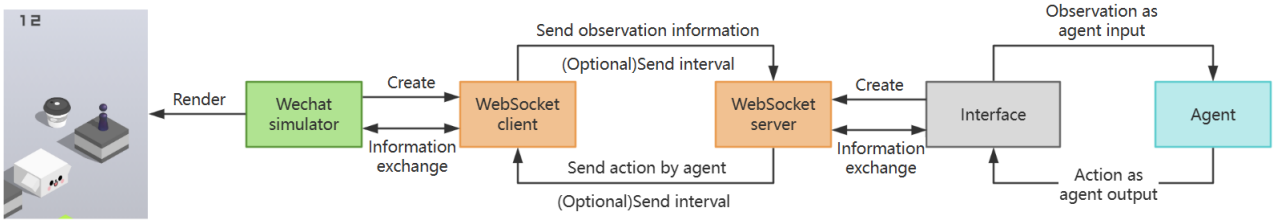


Fig. 2: Overall architecture of WemiEnv

WebSocket server and encapsulates them in a convenient format. The RL agent interacts directly with the interface. It's implemented in Python.

B. Real-time Interactive Framework

WemiEnv adopts a real-time interactive framework based on the WebSocket API. Before formally establishing the WebSocket channel, WemiEnv creates a WebSocket server through the WebSocket communication interface to wait for client connections. Afterward, the WeChat simulator will connect to the pre-created WebSocket server through the WebSocket client. Through this way, a full-duplex communication channel is established between the WemiEnv and the WeChat simulator. Subsequently, WemiEnv receives environmental observations from the simulator through this channel and passes them to the RL agent. The agent evaluates the next action under its policy based on this environmental information and sent to WemiEnv. WemiEnv then transmits the agent's action to the WeChat simulator. After the WeChat simulator performs the corresponding operation in the simulated mini-game, it sends the next set of observation data and the reward signal to the WemiEnv via the WebSocket channel, thus forming a communication loop for the interaction between the RL agent and the mini-game. The detailed interactive timeline between the RL agent, WemiEnv, and the WeChat simulator is shown in Fig. 3.

Due to the real-time interactive property of WemiEnv, the WeChat simulator does not pause the simulation of the mini-game while waiting for the agent to return the next action. In real-world scenarios, the agent may spend time choosing a proper decision, or the action may be delayed during transmission. Meanwhile, the WeChat simulator still runs the game. Therefore, it is mandatory to take the *simulation latency* into consideration when training the RL agent, which starts from the sending of observations and ends with the receiving of the actions of the agent. The simulation latency will be further analyzed in the following section.

Optionally, during the information exchange between the WebSocket server and the client, a parameter containing user-defined latency can also be introduced to the communication channel. This latency can facilitate the setting of frame skip in some RL algorithms when solving video game tasks [12], [11].

TABLE II: A run example of Flappy Bird mini-game task

```
# Create the Flappy Bird environment
env = WemiEnv.create_env(Env_name='Flappybird')
# Initialize the environment
observation = env.reset()
# Execute an action in the environment
observation_, reward, done, info = env.step(action)
```

C. Application Programming Interface

WemiEnv provides an easy-to-use application programming interface (API). For user-customized mini-games, the following functions provide communication from the mini-game to the RL agent.

1) Environment Interface: WemiEnv adopts the OpenAI Gym-style environment interface. An example of a run function is shown in Table II. Users can easily access the WeChat mini-games through the three functions:

- create_env(): Create an environment based on its name.
- reset(): Initialize the environment and return the initial observation.
- step(): Execute an action in the environment and return the next observation, reward, done signal, and information from the environment.

2) WebSocket Server: The Websocket server provides four functions to communicate with the client:

- create_server(): Create a WebSocket server for communication.
- send_message(): Send a specific message to the client.
- recv_message(): Retrieve the most recent message from the client.
- waiting_client(): Wait for the client to connect.

3) WebSocket Client: The WebSocket Client mainly provides two functions to communicate with the server:

- connectSocket(): Create a WebSocket client and connect to a WebSocket server.
- socket_send(): Send a message to the server.

The detailed usage of the Wemi API can be found on our GitHub repositories.

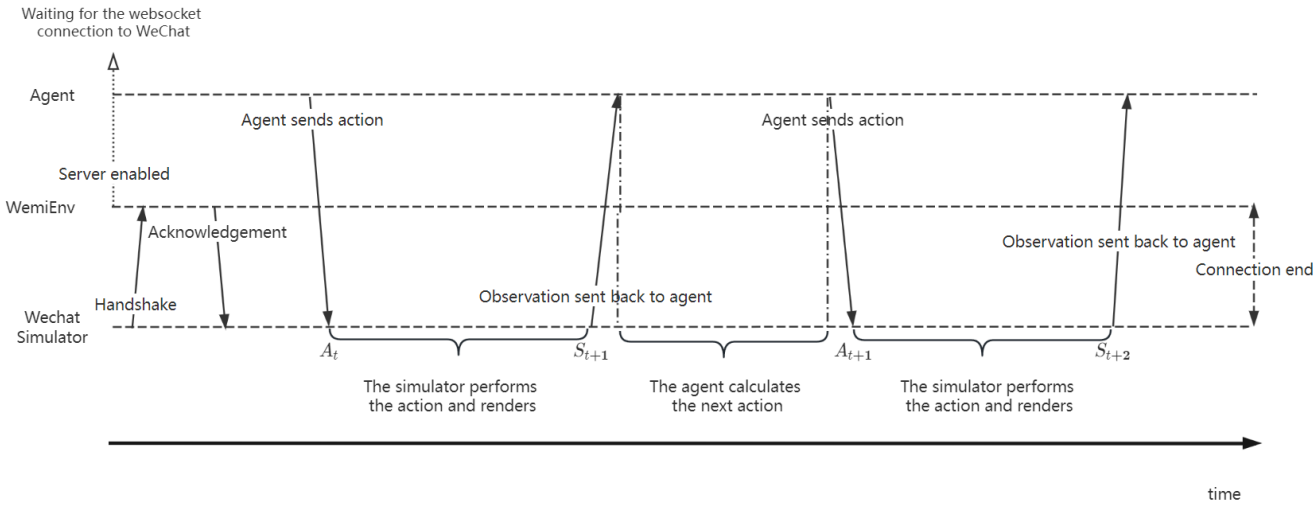


Fig. 3: The interactive timeline between the RL agent and WeChat simulator. The agent observes an initial state S_0 after the environment is initialized. After the agent makes a decision and executes action A_0, the environment transfers to the new state S_1, and this process repeats.

D. Support Tasks

Currently, WemiEnv supports six mini-game tasks:

- **Space Fighter**[8] is a classic arcade-style video game that typically involves controlling a fighter ship to navigate through space while battling against enemy spacecraft and obstacles. The default observation space is a position vector containing the fighter ship and currently available enemies. The action space is a 2-dimensional vector containing the moving direction and distance of the fighter ship.
- **Flip**[9] is a simple yet engaging game that gained popularity as a viral trend, particularly on social media platforms. The game involves flipping a bottle-shaped object and attempting to make it land upright on a flat surface by touching the screen. The release time denotes the flipping distance of the bottle. The default observation space is the distance between two adjacent surfaces and the action space is a float denoting the release time of the bottle which ranges from 300 microseconds to 1200 microseconds.
- **2048**[10] is a popular puzzle game that was created by Gabriele Cirulli in 2014. The game is played on a 4x4 grid, and the objective is to combine numbered tiles to reach the tile with the number 2048. Players can swipe left, right, up, or down on the grid to move all tiles in that direction. The default observation space is the matrix describing the grid. The action space is a set of discrete values containing $\{0,1,2,3\}$, which indicates the swipe direction.
- **Flappy Bird**[11] is a simple yet notoriously difficult mobile game developed by Vietnamese developer Dong Nguyen and released in 2013. The game became immensely popular due to its addictive gameplay and minimalist design. Players control a bird avatar by tapping on the screen and aiming at flying across the obstacles. The default observation space is the position of the bird and the relative position of the nearest obstacles. The action space is a discrete value set containing $\{0,1\}$, which denotes the touch of the screen.
- **Timberman**[12] is a casual arcade game where the player controls a lumberjack character who must chop down an endless tree while avoiding branches. The main goal is to chop down as much of the tree as possible before time runs out or the player makes too many mistakes. The default observation space is a vector that describes the position of the branches. The action space is a discrete value set containing $\{0,1\}$, which denotes the action direction.
- **Snake**[13] is a classic arcade game where the player controls a snake that moves around a bordered screen. The objective is to eat food items that randomly appear on the screen. Each time the snake eats a piece of food, its length increases. The game ends if the snake collides with the screen border or with its own tail. The observation space is the matrix denoting the position of the snake and the food. The action space is a set of discrete values containing $\{0,1,2,3\}$, which indicates the moving direction of the snake.

The screenshots of the six mini-game tasks can be found

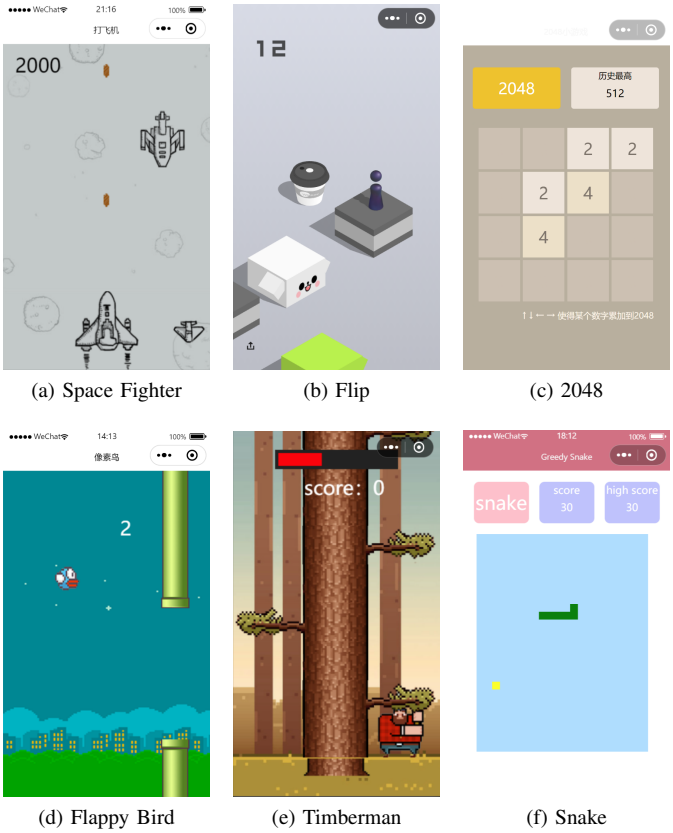
[8]https://github.com/qutz/we_plain
[9]https://github.com/yaoshanliang/weapp-jump
[10]https://github.com/jeffche/wechat-app-2048
[11]https://github.com/qutz/we_flappybird
[12]https://github.com/FishZodiac/crazyTree
[13]https://github.com/Wishing9696/snake

TABLE III: Details of the six ready-to-use mini-game tasks

Task name	Observation space	Action space type	Action space	Comment on action space
Space Fighter	Player-controlled fighter position, number of enemies, position of each enemy.	Continuous	2-D float vector	The moving direction and distance of the fighter.
Flip	Distance between the bottle and the next platform.	Continuous	float number	The release time of the bottle.
2048	4×4 state matrix.	Discrete	{0,1,2,3}	The moving direction of the numbers.
Flappy Bird	The relative distance between the bird and the nearest obstacle pair.	Discrete	{0,1}	Touch or not touch the screen.
Timberman	11×1 state vector denoting the position of all the branches of the tree.	Discrete	{0,1}	The cutting direction of the lumberjack.
Snake	28×22 state matrix denoting the position of the snake and the food.	Discrete	{0,1,2,3}	The moving direction of the snake.

in Fig. 4. Additionally, the details of the six tasks can also be found in Table III. Note that the six mini-game tasks are all based on the open source projects from GitHub. The usage of the WeChat simulator also satisfies the Terms of Service of the WeChat(Wexin) mini-program service terms [24].

Of course, user-customized games can be added to WemiEnv. For new games, a new WemiEnv interface class is required to be implemented. In this class, the reset(), step (), and initialization functions are mandatory. In the initialization process, the communication channel is required to be created using the WebSocket server. Subsequently, the users need to go through the source code of the mini-games and determine the states, rewards, and done signal to be transferred, and then implement information exchange functions with the WebSocket client. Finally, the server will be connected to the client with the above implementations. A detailed demonstration of adding new games is demonstrated in our Github repository.



(a) Space Fighter (b) Flip (c) 2048

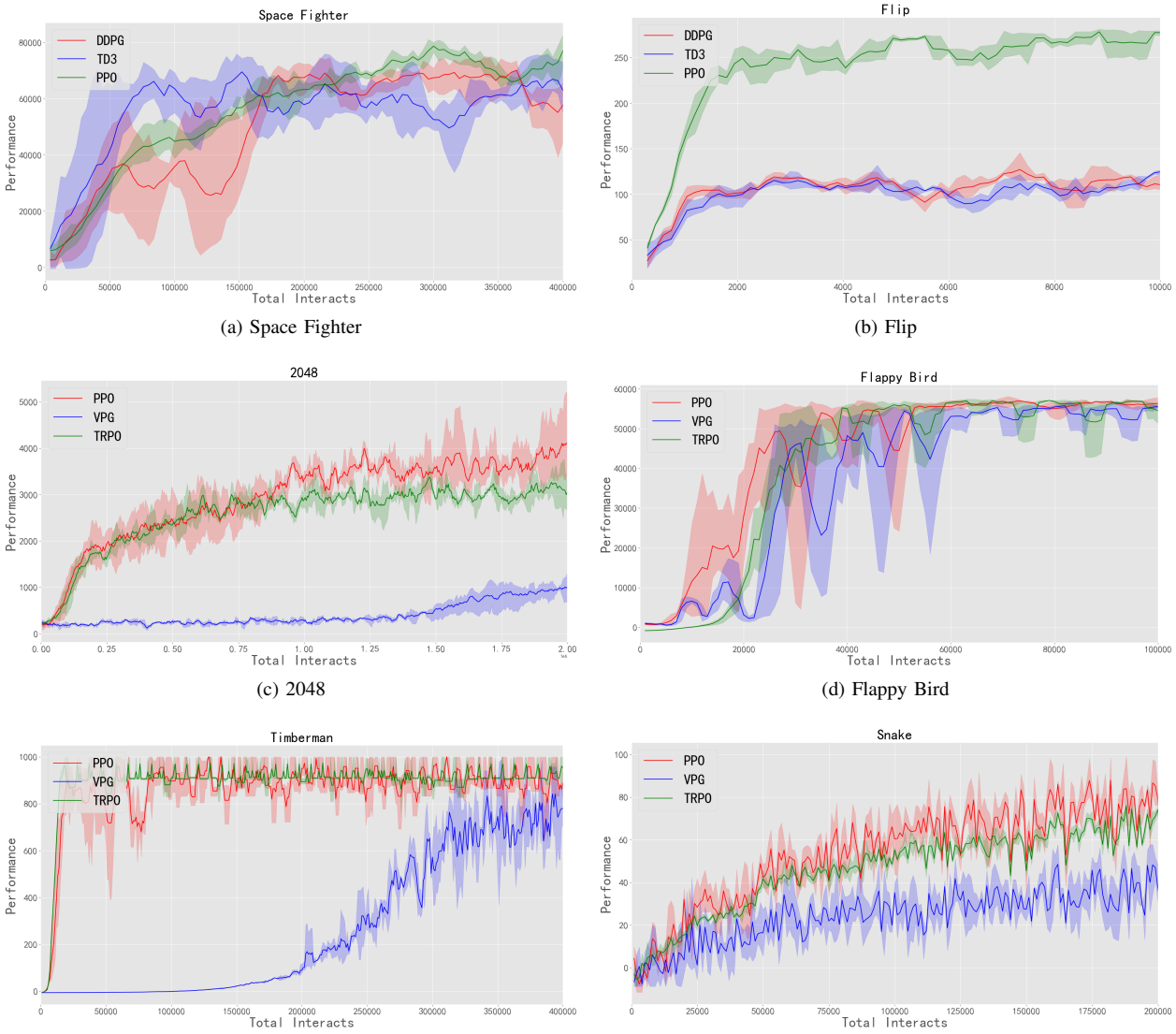
(d) Flappy Bird (e) Timberman (f) Snake

Fig. 4: Screenshots of the six available WeChat mini-games of WemiEnv

E. Latency Analysis

WebSocket communication is almost real-time because it provides full-duplex communication over a single TCP connection. Compared to the traditional HTTP request-response model, WebSocket typically has lower latency. However, despite its low latency, WebSocket communication can still experience significant delays in certain situations. The presence of delays often depends on the following factors:

- **Transmission time**: Transmission time refers to the time it takes for data to be transmitted from the client to the server or vice versa. It depends on factors such as the distance the data packets transmission in the network, the network topology, and the performance of other network devices such as routers and switches. If the user establishes the server and client locally, the transmission time can be almost negligible. However, if the user interacts with a local server through a remote server, the impact of transmission time must be considered.
- **Network congestion**: When a large amount of data flows into a channel and exceeds its capacity, the channel may become congested. This situation may occur due to either a large volume of data being transferred at once or a high transmission frequency. Additionally, packet loss is also a contributing factor to congestion. When data packets are lost or damaged during transmission, the receiving end may request retransmission. If this occurs too frequently, it can also lead to congestion. During congestion, delays tend to increase as communication time prolongs, ultimately severely impacting the normal training of RL agents. In the event of congestion, users can address it by adjusting interface parameters to modify transmission frequency, reducing the size of information exchanged, or optimizing the hardware's network configuration.
- **Network failure**: Network failures can also significantly affect network performance. If the faulty device is a critical component along the path of the data packets (such as routers or switches), it may fail to process



(a) Space Fighter (b) Flip

(c) 2048 (d) Flappy Bird

(e) Timberman (f) Snake

Fig. 5: Performance evaluation of three deep RL algorithms on the six mini-game tasks. The results are averaged with five different runs, and the agent and tasks are deployed on the same device.

or forward packets, resulting in packet loss. In cases of device failure, the network may attempt to bypass the faulty device, increasing the transmission time and latency of the packets.

We also conduct experiments on the 6 tasks to test the latency of the WebSocket communication. The details can be found in the experiment part.

IV. Experiments

In this section, we conduct experiments on WemiEnv with several popular deep RL algorithms. We first describe the experimental setups and then report the experimental results. We intend to answer the following research questions (RQ) through experiments:

- **RQ1**: How do deep RL algorithms perform on the tasks provided by WemiEnv?
- **RQ2**: What about the communication latency when training on the six tasks with deep RL methods?
- **RQ3**: What is the effect of different levels of communication latency on training the RL agent with real-time games?

A. Experimental Setup

1) Tested RL Algorithms: For the six mini-game tasks in WemiEnv, Space Fighter and Flip are tasks with continuous action space, while 2048, Flappy Bird, Timberman and Snake are tasks with discrete action space. Therefore, we use different sets of RL algorithms to run the tasks. We adopt OpenAI SpinningUp RL baseline algorithms for evaluation. For continuous action space tasks, the Deep Deterministic Policy Gradient (DDPG) [11], Twin Delayed Deep Deterministic Policy Gradient (TD3) [7] and Proximal Policy Optimization(PPO) [17] are used. For discrete action space

tasks, Vanilla Policy Gradient (VPG) [22], PPO, Trust Region Policy Optimization (TRPO) [16] are used for evaluation.

2) Neural Network Architecture: For VPG, PPO, and TRPO, both the policy and value networks consist of three layers, each with 64 neurons activated by the ReLU function. In addition, for DDPG, TD3, they adopt 3-layer policy and value networks with 256 neurons per layer. The output layers for the policy network of DDPG and TD3 adopt Tanh as the activation function. The dimensions of the input and output layers are aligned with the environment specifications, and normalization is applied to the inputs to enhance stability.

3) Learning Settings: The hyperparameters of the deep RL methods are mostly selected by using the grid search. The discount factor in our experiment is searched for in $\{0.6, 0.9, 0.99, 1\}$ and we found that $\gamma = 0.99$ achieves the best performance for most methods. For VPG, the learning rates for the actor and critic networks are searched from $\{0.01, 0.001, 0.0001, 0.0003, 0.0005\}$, and the best performance of the learning rates is 0.0003 and 0.001, respectively. The generalized advantage estimation (GAE) factor of VPG λ is searched through $\{0, 0.9, 0.97, 1\}$ and $\lambda = 0.97$ to obtain the best performance. For PPO, the clipping parameter ε is set to 0.2 for all tasks. The learning rates for the actor and critic networks of PPO are searched from $\{0.01, 0.001, 0.0001, 0.0003, 0.0005\}$ and the best performance of the learning rates are 0.0003 and 0.001, respectively. The generalized advantage estimation (GAE) factor of PPO λ is searched through $\{0, 0.9, 0.97, 1\}$ and $\lambda = 0.97$ to obtain the best performance. For TRPO, the KL-divergence constraint is set to 0.01. The learning rates for the TRPO actor and critic networks are searched from $\{0.01, 0.001, 0.0001, 0.0003, 0.0005\}$ and the best performance of the learning rates is 0.01 and 0.01, respectively. The backtracking line search is conducted on 10 conjugate gradient iterations with a maximum of 10 steps and a step fraction of 0.8. For DDPG, the learning rates for the actor and critic networks are searched from $\{0.01, 0.001, 0.0001, 0.0003, 0.0005\}$ and the best learning rate performance is 0.001 and 0.001, respectively. The size of the replay buffer is set to 1,000,000, and the learning is performed by sampling 100 mini-batch samples. The soft update parameter τ is set to 0.995. A Gaussian distribution $N(0, 0.1)$ is added to the output action to ensure exploration. For TD3, the learning rates, size of the replay buffer, and size of mini-batch are the same as DDPG. The action clipping is set to 0.5 to prevent excessive perturbations. The update delay is set to 2. Other hyperparameters of our experiments can be found on our GitHub.

B. Performance Evaluation

In this part, we evaluate the performance of deep RL algorithms with the OpenAI SpinningUp RL library, to show the usability of WemiEnv and to answer RQ1. The experimental results are shown in Fig. 5. The X-axis represents the number of steps that interact with the environment, while the Y-axis represents the total score of each episode. We run each algorithm five times and report the average score for each episode. The shaded region in the figure denotes the variance of five runs. The agent and the tasks are deployed on the same device. The performances of deep RL algorithms vary for the six mini-game tasks. For the six mini-game tasks, PPO performs mostly the best after convergence. VPG performs far behind the other algorithms in Flip, 2048, Timberman and Snake. Specifically, in game 2048, the VPG algorithm tries hard to learn a proper policy. For FlappyBird, the median scores for the top 40 human players are 133.5[14]. The performance of RL agent in FlappyBird surpasses the performance of human players. The experimental results show that the deep RL methods from the OpenAI Spinning Up library can be properly trained on the initial tasks provided by WemiEnv. Therefore, WemiEnv is capable of serving as a benchmark environment for deep RL algorithms.

TABLE IV: Communication latency of the six tasks

Task	Local host latency	Remote host latency
Space fighter	16ms	18ms
Flip	2ms	5ms
2048	3ms	6ms
Flappy Bird	2ms	5ms
Timberman	3ms	5ms
Snake	6ms	8ms

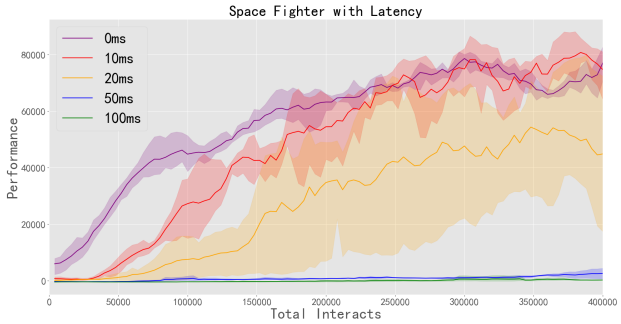
C. Latency Evaluation

In this section, our objective is to evaluate the communication of latencies of different tasks to answer RQ2. We evaluated the communication latency of WemiEnv in two settings: 1) Both client and server are deployed on the same computer (local host setting). 2) The client and the server are deployed on different computers and communicate through a LAN (remote host). For both settings, the experiments were performed on an AMD Ryzen 7 5800H 3.2GHz laptop GPU with NVidia GeForce RTX 3060 GPU. The communication latency is depicted in Table IV. The results of the six tasks show that WemiEnv's communication latency is relatively small. In addition, we observe that the latency is related to the size of the transmission state. For the games Space Fighter and Snake, the state is larger than the other four games, leading to larger transmission latencies. In the next section, we will evaluate the effect of the manually added latencies on two real-time games: Space Fighter and Flappy Bird.

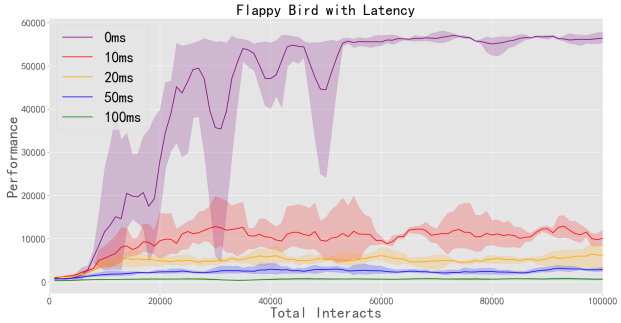
D. Influence of Communication Latency

In this section, we aim to show the effect of communication latencies when training the agents with tasks provided by WemiEnv, in order to answer the RQ3. We evaluate the influence of communication latency on two real-time games: Space Fighter and Flappy Bird. We manually add latency in WemiEnv to simulate the underlying latency in real-world network settings. We specifically evaluate the performance of PPO, as it achieved considerable performance on the two tasks. The artificially introduced delays are set to 10ms, 20ms, 50ms, 100ms. In addition, the performance of no artificial delay is also compared. The results are depicted in Fig 6. For Space

[14]https://flappybird.io/leaderboard/



(a) Space Fighter



(b) Flappy Bird

Fig. 6: Performance evaluation of PPO under different settings of Latency

Fighter, the performances drop when the latency is set to 10ms and 20ms. When the latency grows over 50ms, the RL agent cannot even learn a suitable policy. For Flappy Bird, it is more sensitive to the latency involved. Performance deteriorates rapidly as latency increases. The results indicate that the training of the two real-time games can be affected when there is significant communication latency. Therefore, in a complex network scenario, latency should be carefully considered when training RL agents with WemiEnv in practice. We recommend that the user measure the network latency before training the RL agents, to avoid the influence of communication delays.

V. Conclusion

In this paper, we propose WemiEnv, which is designed for the RL benchmark environment on WeChat mini-games. WemiEnv is easy-to-use, lightweight and efficient with low communication latency. The WemiEnv API allows user-customed mini-games by simply implementing several interface functions. In addition, we provide an initial set of ready-to-use mini-game tasks implemented with WemiEnv. Experimental results with several open-source RL baseline algorithms demonstrated the usage of WemiEnv. One issue for WemiEnv is that the mini-games are required to run on the WeChat simulator, which is deployed on the computer. In the future, we will develop the mobile version of WemiEnv to support games running on the mobile phone. In addition, we will continue to implement more ready-to-use mini-game tasks in WemiEnv.

Acknowledgements

This research was funded by the National Science and Technology Major Project (2022ZD0119103). This paper is supported by the Fundamental Research Funds of Hangzhou City University (No. X-202306). This research is also supported by the advanced computing resources provided by the Supercomputing Center of Hangzhou City University.

Appendix

Table V illustrates some of the available open-sourced WeChat mini-games. With WemiEnv, those mini-games can be easily utilized to train the RL agents.

References

[1] Bharathan Balaji, Sunil Mallya, Sahika Genc, Saurabh Gupta, Leo Dirac, Vineet Khare, Gourav Roy, Tao Sun, Yunzhe Tao, Brian Townsend, et al. Deepracer: Educational autonomous racing platform for experimentation with sim2real reinforcement learning. *arXiv preprint arXiv:1911.01562*, 2019.

[2] Bharathan Balaji, Sunil Mallya, Sahika Genc, Saurabh Gupta, Leo Dirac, Vineet Khare, Gourav Roy, Tao Sun, Yunzhe Tao, Brian Townsend, et al. Deepracer: Autonomous racing platform for experimentation with sim2real reinforcement learning. In *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pages 2746–2754. IEEE, 2020.

[3] Charles Beattie, Joel Z Leibo, Denis Teplyashin, Tom Ward, Marcus Wainwright, Heinrich Küttler, Andrew Lefrancq, Simon Green, Víctor Valdés, Amir Sadik, et al. Deepmind lab. *arXiv preprint arXiv:1612.03801*, 2016.

[4] Marc G Bellemare, Yavar Naddaf, Joel Veness, and Michael Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47:253–279, 2013.

[5] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym. *arXiv preprint arXiv:1606.01540*, 2016.

[6] Chuqiao Chen, Fugen Yao, Dong Mo, Jiangtao Zhu, and Xiqun Michael Chen. Spatial-temporal pricing for ride-sourcing platform with reinforcement learning. *Transportation Research Part C: Emerging Technologies*, 130:103272, 2021.

[7] Scott Fujimoto, Herke Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In *International conference on machine learning*, pages 1587–1596. PMLR, 2018.

[8] Matthew Johnson, Katja Hofmann, Tim Hutton, and David Bignell. The malmo platform for artificial intelligence experimentation. In *Ijcai*, pages 4246–4247, 2016.

[9] Arthur Juliani, Vincent-Pierre Berges, Ervin Teng, Andrew Cohen, Jonathan Harper, Chris Elion, Chris Goy, Yuan Gao, Hunter Henry, Marwan Mattar, and Danny Lange. Unity: A general platform for intelligent agents. *arXiv preprint arXiv:1809.02627*, 2020.

[10] Michał Kempka, Marek Wydmuch, Grzegorz Runc, Jakub Toczek, and Wojciech Jaśkowski. Vizdoom: A doom-based ai research platform for visual reinforcement learning. In *2016 IEEE conference on computational intelligence and games (CIG)*, pages 1–8. IEEE, 2016.

[11] T Lilicrap, J Hunt, Alexander Pritzel, N Hess, Tom Erez, D Silver, Y Tassa, and D Wiestra. Continuous control with deep reinforcement learning. In *International Conference on Representation Learning (ICRL)*, 2016.

[12] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *nature*, 518(7540):529–533, 2015.

[13] Matej Moravčík, Martin Schmid, Neil Burch, Viliam Lisỳ, Dustin Morrill, Nolan Bard, Trevor Davis, Kevin Waugh, Michael Johanson, and Michael Bowling. Deepstack: Expert-level artificial intelligence in heads-up no-limit poker. *Science*, 356(6337):508–513, 2017.

[14] Adrian Remonda, Nicklas Hansen, Ayoub Raji, Nicola Musiu, Marko Bertogna, Eduardo Veas, and Xiaolong Wang. A simulation benchmark for autonomous racing with large-scale human data. *arXiv preprint arXiv:2407.16680*, 2024.

[15] Pierre Schegg, Etienne Ménager, Elie Khairallah, Damien Marchal, Jérémie Dequidt, Philippe Preux, and Christian Duriez. Sofagym: An open platform for reinforcement learning based on soft robot simulations. *Soft Robotics*, 10(2):410–430, 2023.

TABLE V: Open-sourced examples of WeChat mini-games

Game	Genre	GitHub
Space Fighter	Action	https://github.com/tiancheng2000/WeChatFighter
FlappyBird	Casual	https://github.com/Ezharjan/FlappyBird
Tetris	Puzzle	https://github.com/bofeiw/Tetris-Wechat
Paper Cuts	Puzzle	https://github.com/infatuationX/papercuts-master
JumpAndJump	Casual	https://github.com/zouhuidong/JumpAndJump
ColorRealm	Casual	https://github.com/squa7ed/ColorRealm
Life Swings	Casual	https://github.com/pearmini/life-swings
WechatAirplane	Action	https://github.com/cpxiao/WechatAirplane
Landscape Painting Puzzle	Puzzle	https://github.com/Y-A-K-E/landscape_painting_sliding_puzzle
ShuleShule	Casual	https://github.com/biggermelon3/ShuleShule
Escape	Action	https://github.com/Makcy/escape
Whac-A-Mole	Casual	https://github.com/Edwin-Xu/miniGame
AllOrNothing	Puzzle	https://github.com/aillear/AllOrNothing
Run-and-Jump	Adventure	https://github.com/YTGhost/Run-and-Jump
Flying Bird	Casual	https://github.com/yang1015/WeChat-MiniGame
Tank Battle	Stategy	https://github.com/Aimee1608/wechatGame-all
Find the Difference	Puzzle	https://github.com/Aimee1608/wechatGame-all
Link-up	Puzzle	https://github.com/Aimee1608/wechatGame-all
Angry birds	Casual	https://github.com/Aimee1608/wechatGame-all
Jigsaw Puzzle	Puzzle	https://github.com/nanwangjkl/sliding_puzzle
Bear Jump	Casual	https://github.com/charlieysx/bearjump
Sheep-And-Sheep	Puzzle	https://github.com/P1kaj1uu/Wechat-Sheep-And-Sheep
ColorUp	Casual	https://github.com/shichaohui/ColorUp
Synthesis Watermelon	Puzzle	https://github.com/Degree-21/synthesis_watermelon
Finger Adventure	Role-play game	https://github.com/viczhou/WX_Game
Yutu's Treasure	Role-play game	https://github.com/pfan123/YutuTreasure
3D Racer	Race	https://github.com/superman-t/Racer
Basketball	Sports	https://github.com/Pyloons/wxMiniGame-basketballDemo
Land Lord	Card game	https://github.com/Zhang19910325/nodejs-server-wechat-landLordGame
Gobang	Card game	https://github.com/ObservedObserver/Wechat-LittleGame
Three Kingdoms	Strategy	https://github.com/duanfengzhi/A-WeChat-Game-Of-Three-Kingdoms-For-Territorial-Supremacy-BC
Hear and Remember	Puzzle	https://github.com/owlhpy/wechat-game
Poem Challenge	Puzzle	https://github.com/owlhpy/wechat-game
Fishing Cat	Sports	https://github.com/ayarami/examples-game-FishingCat

[16] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *International conference on machine learning*, pages 1889–1897. PMLR, 2015.

[17] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[18] Longxiang Shi, Shijian Li, Longbing Cao, Long Yang, and Gang Pan. Tbq (σ) improving efficiency of trace utilization for off-policy reinforcement learning. In *Proceedings of the 18th International Conference on Autonomous Agents and MultiAgent Systems*, pages 1025–1032, 2019.

[19] Tianlin Shi, Andrej Karpathy, Linxi Fan, Jonathan Hernandez, and Percy Liang. World of bits: An open-domain platform for web-based agents. In *International Conference on Machine Learning*, pages 3135–3144. PMLR, 2017.

[20] David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *nature*, 529(7587):484–489, 2016.

[21] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the game of go without human knowledge. *nature*, 550(7676):354–359, 2017.

[22] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018.

[23] Gabriel Synnaeve, Nantas Nardelli, Alex Auvolat, Soumith Chintala, Timothée Lacroix, Zeming Lin, Florian Richoux, and Nicolas Usunier. Torchcraft: a library for machine learning research on real-time strategy games. *arXiv preprint arXiv:1611.00625*, 2016.

[24] Tencent. Wechat (wexin) mini-program platform-terms of service. *https://developers.weixin.qq.com/miniprogram/en/product/service.html*.

[25] Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ international conference on intelligent robots and systems*, pages 5026–5033. IEEE, 2012.

[26] Daniel Toyama, Philippe Hamel, Anita Gergely, Gheorghe Comanici, Amelia Glaese, Zafarali Ahmed, Tyler Jackson, Shibl Mourad, and Doina Precup. Androidenv: A reinforcement learning platform for android. *arXiv preprint arXiv:2105.13231*, 2021.

[27] Saran Tunyasuvunakool, Alistair Muldal, Yotam Doron, Siqi Liu, Steven Bohez, Josh Merel, Tom Erez, Timothy Lillicrap, Nicolas Heess, and Yuval Tassa. dm_control: Software and tasks for continuous control. *Software Impacts*, 6:100022, 2020.

[28] Oriol Vinyals, Timo Ewalds, Sergey Bartunov, Petko Georgiev, Alexander Sasha Vezhnevets, Michelle Yeo, Alireza Makhzani, Heinrich Küttler, John Agapiou, Julian Schrittwieser, et al. Starcraft ii: A new challenge for reinforcement learning. *arXiv preprint arXiv:1708.04782*, 2017.

[29] Chaozheng Wang, Haochuan Lu, Cuiyun Gao, Zongjie Li, Ting Xiong, and Yuetang Deng. A unified framework for mini-game testing: Experience on wechat. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 1623–1634, 2023.

[30] Qiulei Wang, Lei Yan, Gang Hu, Chao Li, Yiqing Xiao, Hao Xiong, Jean Rabault, and Bernd R Noack. Drlinfluids: An open-source python platform of coupling deep reinforcement learning and openfoam. *Physics of Fluids*, 34(8), 2022.

[31] Jiaqi Xu, Bin Li, Bo Lu, Yun-Hui Liu, Qi Dou, and Pheng-Ann Heng. Surrol: An open-source reinforcement learning centered and dvrk compatible platform for surgical robot learning. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1821–1828. IEEE, 2021.

[32] Daochen Zha, Kwei-Herng Lai, Yuanpu Cao, Songyi Huang, Ruzhe Wei, Junyu Guo, and Xia Hu. Rlcard: A toolkit for reinforcement learning in card games. *arXiv preprint arXiv:1910.04376*, 2019.

[33] Lianmin Zheng, Jiacheng Yang, Han Cai, Ming Zhou, Weinan Zhang, Jun Wang, and Yong Yu. Magent: A many-agent reinforcement learning platform for artificial collective intelligence. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.



Longxiang Shi received Ph.D degree in computer science from Zhejiang University in 2020. In 2018, he was a Joint Ph.D. Student with the Advanced Analytics Institute, University of Technology Sydney. He is currently a lecturer at the Department of Computer Science and Computing at Hangzhou City University. His research interests include reinforcement learning, recommender systems, data mining, and machine learning.



Qianchen Ding is pursuing a bachelor's degree with the Department of Computer Science and Computing at Hangzhou City University, China.



Jingzhe Hou is pursuing a bachelor's degree with the Department of Computer Science and Computing at Hangzhou City University, China.



Binbin Zhou received the B.Math. degree from Hangzhou Normal University in 2008, the M.Phil degree in computer science from Hongkong Polytechnic University in 2011, and the Ph.D. degree in computer science from Zhejiang University in 2021. She is currently an Associate Professor at the Department of Computer Science and Computing, Hangzhou City University. Her research interests are mainly in urban computing, brain-inspired computing, and ubiquitous computing.



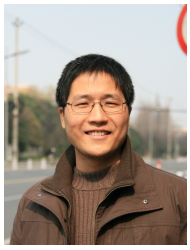
Canghong Jin is an associate professor of computer science at Hangzhou City University. His research focuses on mining and modeling large social and information networks, spatio-temporal series mining, and Big data platforms. Problems he investigated are motivated by large-scale transit records, the web, and online media.



Ye Tao is an Associate Professor in the Industrial Design Department at Hangzhou City University. Her research focuses on intelligent design and human-computer interaction.



Jinling Wei is a professor at Hangzhou City University. He got his Ph.D degree in industrial automation from Zhejiang University in 1999. His main research interests include artificial intelligence, big data platforms, software engineering, and information management.



Shijian Li is a full professor at the College of Computer Science, Zhejiang University. He earned his Ph.D. degree from the same college in 2006 and worked as a Postdoctoral Fellow in the Department of Control Science and Engineering at Zhejiang University from 2006 to 2008. His research interests include Ubiquitous Computing, artificial intelligence, and human-computer interaction, and has published more than 70 papers on these topics.